

Harness Engineering 最佳实践：从概念到落地的完整操作手册



AI4S
Stay hungry, stay foolish

关注他

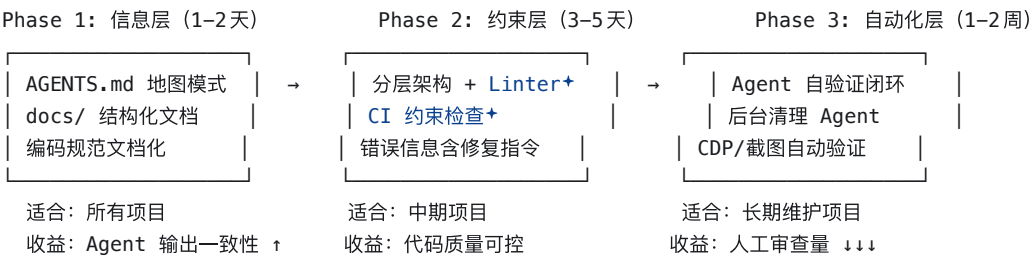
收录于 · AIGC >

56 人赞同了该文章 >

本文是 [Harness Engineering 上篇](#) 的续篇。上篇解读了 [OpenAI+](#) 的方法论——什么是 Harness Engineering、核心理念、五大工程实践和哲学内核。本篇不再重复概念，直接进入实操：怎么在你自己的项目中落地 Harness Engineering，用什么工具，踩什么坑，有哪些可以直接抄的模板。

一、落地路线图：三步走

在动手之前，先画清楚路线。Harness Engineering 的落地不是"一把梭"，而是渐进式的：



不要一步到位。很多人失败就在于想一次性搭完所有基础设施。Phase 1 已经能带来显著提升，Phase 2 是质变点，Phase 3 是锦上添花。

二、Phase 1：信息层——让 Agent "看得懂"你的项目

2.1 AGENTS.md：写地图，不写百科全书

上篇提到 OpenAI 用"地图模式"替代超长指令文件。这里给出可以直接拷贝使用的模板。

反面教材（别这么写）：

```
# ❌ 错误示范：把所有内容塞进一个文件
我们使用 React 18 + TypeScript 5.3 + Next.js 14...
组件命名使用 PascalCase, 函数用 camelCase...
数据库用 PostgreSQL+, ORM 用 Prisma+...
(后面还有 500 行)
```

问题：挤占上下文窗口、难以维护、Agent 很难定位需要的信息。

正确做法（直接用）：



```
# AGENTS.md
```

项目简介

[一句话] 这是一个面向中小企业的在线项目管理平台，基于 Next.js + PostgreSQL。

快速导航

```
| 你想做什么 | 去哪里看 |  
|-----|-----|  
| 了解系统架构 | docs/architecture/overview.md |  
| 了解模块边界和依赖规则 | docs/architecture/boundaries.md |  
| 了解编码规范 | docs/conventions/README.md |  
| 了解当前迭代任务 | docs/plans/current-sprint.md |  
| 了解 API 规范 | docs/reference/api-spec.yaml |  
| 了解错误码 | docs/reference/error-codes.md |  
| 了解测试规范 | docs/conventions/testing.md |
```

硬性规则（必须遵守，CI 会验证）

1. 依赖方向: types/ → config/ → repo/ → service/ → runtime/ → ui/
2. 横切关注点 (auth/log/telemetry) 只能通过 Provider 注入
3. 单文件不超过 300 行
4. 新增代码必须有对应测试
5. 使用结构化日志，禁止 console.log

提交规范

- feat: 新功能
- fix: 修复
- refactor: 重构
- docs: 文档
- test: 测试

关键设计原则：

- • AGENTS.md 控制在 50-100 行。超过就说明你在写百科全书了
- • **“你想做什么 → 去哪里看”比“这是什么”更有效——面向任务而非面向知识**
- • 硬性规则单独列出，这些是 CI 会强制验证的，不是“建议”

2.2 docs/ 目录：结构化知识库

```
docs/  
├── architecture/           # 稳定层（很少变）  
│   ├── overview.md        # 系统架构图 + 一段话描述  
│   ├── boundaries.md      # 模块边界和依赖规则  
│   └── data-flow.md        # 数据流转图  
├── conventions/           # 规范层（偶尔更新）  
│   ├── README.md          # 规范总览（索引）  
│   ├── naming.md          # 命名规范  
│   ├── error-handling.md  # 错误处理规范  
│   ├── testing.md         # 测试规范  
│   └── logging.md         # 日志规范  
├── design/                # 设计层（按功能组织）  
│   ├── feature-auth.md    # Status: ✅ Implemented  
│   ├── feature-search.md  # Status: 📋 Approved  
│   └── feature-billing.md  # Status: 📝 Draft  
├── plans/                 # 计划层（频繁变）  
│   ├── current-sprint.md  # 当前迭代  
│   └── backlog.md         # 待办  
└── reference/             # 参考层（自动生成）
```

```
├─ api-spec.yaml
└─ error-codes.md
```

每个文档的头部都加上元信息：

```
---
last_updated: 2026-03-28
status: active          # active | deprecated | draft
owner: @zhangsan
---
```

这样 doc-gardening Agent 可以自动扫描过期文档。

2.3 设计文档模板

Agent 执行复杂功能前，先写设计文档。以下是模板：

```
# Feature: [功能名称]

## Status: 🟡 Draft | 🟢 Approved | 🟠 In Progress | ✅ Implemented

## 目标
一句话描述这个功能要解决什么问题。

## 非目标
明确列出这次不做什么（防止 Agent 扩大范围）。

## 技术方案

### 涉及的模块
- types/: 新增 XXX 类型定义
- service/: 新增 XXX 业务逻辑
- ui/: 新增 XXX 页面

### 数据模型变更
```sql
-- 如有数据库变更，写在这里
```

## API 变更

```
POST /api/xxx
Request: { ... }
Response: { ... }
```

## 验收标准

- 标准 1: 具体的、可验证的
- 标准 2: 具体的、可验证的
- 测试覆盖率  $\geq 80\%$

## 依赖

- 依赖 feature-auth (已实现 ✅)

**\*\*为什么要这么做\*\*:** Agent 拿到一个功能需求后，先填写这个模板（或人工填写），审批通过后再动手写代码。这就是"明确意图"的工程化实现。

```

```

## 三、Phase 2：约束层——让 Agent "不得不"写好代码

### 3.1 分层架构：用 ESLint+Linter 锁死依赖方向

上篇讲了 Types → Config → Repo → Service → Runtime → UI 的六层架构。这里给出具体的 Linter 配置。

#### ESLint 配置 (TypeScript/JavaScript 项目)

```
```.javascript
// eslint.config.js
export default [
  {
    rules: {
      'no-restricted-imports': ['error', {
        patterns: [
          {
            group: ['../../repo/*', '../../repo'],
            message: '❌ UI 层不能直接引用 Repo 层。✅ FIX: 通过 Runtime 层访问数据。📖 See: docs/architecture/',
          },
          {
            group: ['../../service/*', '../../service'],
            message: '❌ UI 层不能直接引用 Service 层。✅ FIX: 通过 Runtime 层暴露的接口访问。📖 See: docs/archi',
          }
        ]
      }
    },
  },
  {
    'no-console': ['error', {
      allow: ['warn', 'error']
    }],
  },
  {
    'max-lines': ['error', {
      max: 300,
      skipBlankLines: true,
      skipComments: true
    }],
  },
  {
    'max-lines-per-function': ['error', {
      max: 50,
      skipBlankLines: true,
      skipComments: true
    }],
  }
];
```

Go 项目：用 go-architect 或自定义脚本

```
#!/bin/bash
# scripts/check-layer-deps.sh
# 检查分层依赖方向

ERRORS=0

# UI 层不能引用 Repo 层
if grep -r '".*\repo"' ./internal/ui/ 2>/dev/null; then
  echo "❌ ERROR: ui/ 层直接引用了 repo/ 层"
  echo "✅ FIX: 通过 runtime/ 层的接口访问数据"
  echo "📖 See: docs/architecture/boundaries.md"
  ERRORS=$((ERRORS + 1))
fi

# Repo 层不能引用 Service 层 (反向依赖)
if grep -r '".*\service"' ./internal/repo/ 2>/dev/null; then
  echo "❌ ERROR: repo/ 层反向引用了 service/ 层"
  echo "✅ FIX: 使用接口 (interface) 解耦, 依赖注入"
```

```

    ERRORS=$((ERRORS + 1))
fi

if [ $ERRORS -gt 0 ]; then
    echo "发现 $ERRORS 个架构违规，请修复后重试。"
    exit 1
fi

echo "✅ 架构依赖检查通过"

```

Python 项目：用 import-linter

```

# .importlinter
[importlinter]
root_packages = myapp

[importlinter:contract:layers]
name = Layer Dependencies
type = layers
layers =
    myapp.ui
    myapp.runtime
    myapp.service
    myapp.repo
    myapp.config
    myapp.types
pip install import-linter
lint-imports # CI 中运行

```

3.2 自定义 Linter 规则：错误信息即 Prompt

这是 Harness Engineering 最有杠杆的实践之一。核心思路：**每条 Linter 报错都必须包含三要素——问题是什么、怎么修、去哪看文档。**

以 ESLint 自定义规则为例，禁止直接使用 `fetch`（要求通过统一的 API Client）：

```

// eslint-rules/no-raw-fetch.js
module.exports = {
  meta: {
    type: 'problem',
    docs: {
      description: 'Disallow raw fetch(), use apiClient instead'
    },
    messages: {
      noRawFetch: [
        '❌ 禁止直接使用 fetch()。',
        '✅ FIX: 使用统一的 API Client: ',
        'import { apiClient } from "@lib/api-client";',
        'const data = await apiClient.get("/endpoint");',
        '📖 See: docs/conventions/api-calls.md'
      ].join('\n')
    }
  },
  create(context) {
    return {
      CallExpression(node) {
        if (node.callee.name === 'fetch') {
          context.report({ node, messageId: 'noRawFetch' });
        }
      }
    }
  }
};

```

```

    }
  };

```

一个万能公式：

- ✖ [什么错了]
- ✔ FIX: [怎么改, 给出代码片段]
- 📖 See: [哪个文档有详细说明]

Agent 看到这种报错，不需要任何额外提示就能自动修复。你写的每一条 Linter 规则，本质上都是一个自动触发的 Prompt。

3.3 CI 管线配置：完整的 Agent 护栏

```

# .github/workflows/agent-guardrails.yml
name: Agent Guardrails

on: [pull_request]

jobs:
  quality-gates:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      # — Gate 1: 类型检查 —
      - name: TypeScript Check
        run: npx tsc --noEmit

      # — Gate 2: Linter (含自定义规则) —
      - name: Lint
        run: npm run lint

      # — Gate 3: 单元测试 —
      - name: Unit Tests
        run: npm test -- --coverage

      # — Gate 4: 覆盖率阈值 —
      - name: Coverage Threshold
        run: |
          COVERAGE=$(npx nyc report --reporter=text-summary | grep 'Lines' | awk '{print $3}' | tr -d '%')
          if (( $(echo "$COVERAGE < 80" | bc -l) )); then
            echo "✖ 代码覆盖率 ${COVERAGE}% < 80%"
            echo "✔ FIX: 为新增代码添加测试"
            exit 1
          fi

      # — Gate 5: 架构约束 —
      - name: Architecture Check
        run: bash scripts/check-layer-deps.sh

      # — Gate 6: 文件大小检查 —
      - name: File Size Check
        run: |
          find src/ -name '*.ts' -o -name '*.tsx' | while read f; do
            lines=$(wc -l < "$f")
            if [ "$lines" -gt 300 ]; then

```

```
echo "❌ $f 有 $lines 行 (上限 300) "
echo "✅ FIX: 拆分为更小的模块, 将辅助函数移至 utils/"
exit 1
fi
done

# — Gate 7: 文档新鲜度 —
- name: Doc Freshness
run: |
  find docs/design/ -name '*.md' | while read f; do
    last_mod=$(git log -1 --format=%ct "$f")
    now=$(date +%s)
    days_old=$(( (now - last_mod) / 86400 ))
    if [ "$days_old" -gt 60 ]; then
      echo "⚠️ $f 已 ${days_old} 天未更新, 可能已过期"
    fi
  done
```

3.4 把主观品味翻译成机械规则

这是 Phase 2 中最需要花时间的部分——把团队的"代码审美"变成 CI 能检查的规则。

团队口头约定	机械化规则	实现方式
"函数要短"	单函数 ≤ 50 行	max-lines-per-function
"文件要短"	单文件 ≤ 300 行	max-lines
"别用 any"	禁止 TypeScript any	@typescript-eslint/no-explicit-any
"日志要规范"	禁止 console.log	no-console
"API 调用要统一"	禁止裸 fetch()	自定义规则 no-raw-fetch
"错误要有上下文"	Error 必须包含 code 字段	自定义规则
"组件要纯"	UI 组件不能直接调数据库	no-restricted-imports
"测试要充分"	覆盖率 ≥ 80%	CI coverage check

经验法则：如果一条规则在 Code Review 中被提过 3 次以上，就应该写成 Linter 规则。

四、Phase 3：自动化层——让 Agent 自我验证和自我修复

4.1 后台清理 Agent：定时任务模板

以下是一个实用的"代码卫生"清理 Prompt，可以配合任何 Agent 工具（Claude Code、Cline、Aider）定期执行：

```
# 任务：代码库卫生清理

请执行以下检查，对每个发现的问题生成独立的修复 PR：

## 检查清单
1. **超长文件**：找出 src/ 下超过 300 行的 .ts/.tsx 文件，拆分为更小模块
2. **缺失测试**：找出 src/ 下没有对应 .test.* 文件的模块，补充基础测试
3. **未使用的 import**：清理所有未使用的 import 语句
4. **TODO/FIXME**：列出所有 TODO 和 FIXME，如果超过 30 天未处理则生成清理 PR
5. **重复代码**：找出高度相似的代码段 (>10行)，提取为共享工具函数
6. **过时文档**：检查 docs/design/ 中状态为 Draft 但已超过 30 天的文档

## 约束
- 每个修复作为独立 PR，不要混在一起
- 每个 PR 修改后必须确保所有测试通过
```

- PR 标题格式：`chore(cleanup): [具体描述]`
- 如果不确定某个修改是否安全，跳过并在 PR 中标注原因

4.2 Git Worktree 自动化脚本

让 Agent 在隔离环境中验证自己的代码：

```
#!/bin/bash
# scripts/agent-verify.sh
# 为指定 PR 分支创建隔离验证环境

BRANCH=$1
WORKTREE_DIR="/tmp/agent-verify-$(date +%s)"

echo "🔧 创建 worktree: $WORKTREE_DIR"
git worktree add "$WORKTREE_DIR" "$BRANCH"

cd "$WORKTREE_DIR"

echo "📦 安装依赖..."
npm ci --silent

echo "🔍 运行类型检查..."
npx tsc --noEmit || { echo "❌ 类型检查失败"; exit 1; }

echo "🔍 运行 Linter..."
npm run lint || { echo "❌ Lint 失败"; exit 1; }

echo "✅ 运行测试..."
npm test || { echo "❌ 测试失败"; exit 1; }

echo "🚀 启动应用验证..."
npm run dev &
DEV_PID=$!
sleep 10

# 基础健康检查
if curl -s http://localhost:3000/api/health | grep -q '"ok"'; then
  echo "✅ 健康检查通过"
else
  echo "❌ 应用启动失败或健康检查未通过"
  kill $DEV_PID 2>/dev/null
  exit 1
fi

kill $DEV_PID 2>/dev/null

echo "🧹 清理 worktree..."
cd -
git worktree remove "$WORKTREE_DIR"

echo "✅ 所有验证通过"
```

4.3 可观测性接入：让 Agent 能"看日志"

在本地开发环境部署一个轻量级的可观测性堆栈，让 Agent 在排错时有据可查：

```
# docker-compose.observability.yml
version: '3.8'

services:
  # 日志: Loki+ + Promtail+
```



```
loki:
  image: grafana/loki:2.9.0
  ports: ["3100:3100"]

promtail:
  image: grafana/promtail:2.9.0
  volumes:
    - ./logs:/var/log/app
    - ./promtail-config.yml:/etc/promtail/config.yml

# 指标: Prometheus
prometheus:
  image: prom/prometheus:latest
  ports: ["9090:9090"]
  volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml

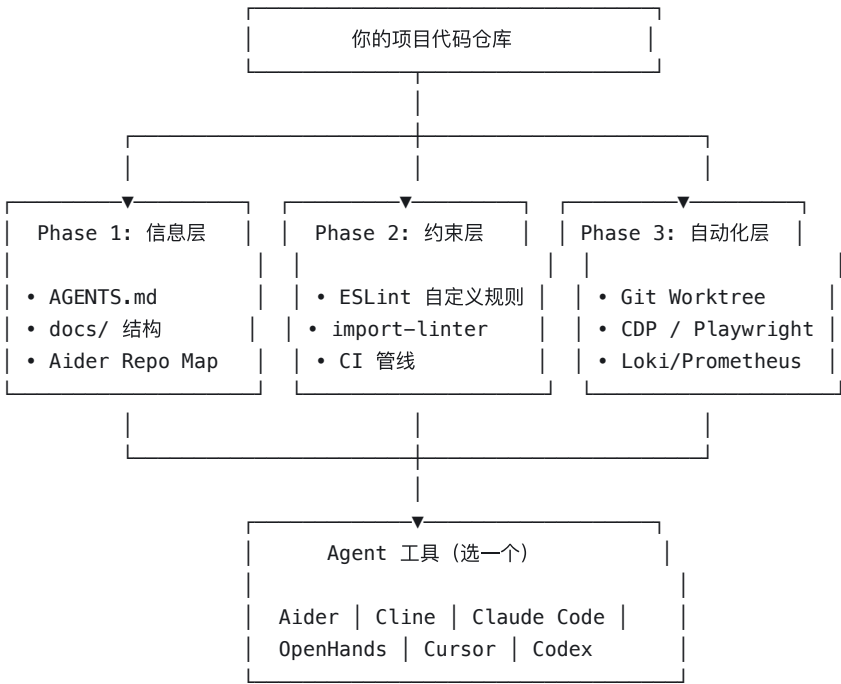
# 可视化: Grafana
grafana:
  image: grafana/grafana:latest
  ports: ["3001:3000"]
  depends_on: [loki, prometheus]
```

Agent 排错时可以用这样的 Prompt：

应用在 /api/users 端点返回 500 错误。
请查看 Loki 日志 (http://localhost:3100) 中最近 5 分钟的错误日志，
找出根因并修复。修复后重新运行对应的测试验证。

五、开源工具推荐：按落地阶段选择

5.1 工具全景图



5.2 Agent 工具对比与选型

工具	Stars	类型	最适合的场景	Harness 友好度
Aider	30k+	CLI 结对编程	个人开发者，终端党	★★★★★
Cline	40k+	VS Code Agent	小团队，需要 Plan/Act 模式	★★★★★
Claude Code	-	CLI Agent	深度自主任务（6h+连续工作）	★★★★★
OpenHands	50k+	平台	团队级部署，需要沙箱隔离	★★★★★
SWE-agent	15k+	CLI Agent	自动修复 GitHub Issue	★★★☆☆
Superpowers	127k	技能框架	强制 TDD + 子 Agent 模式	★★★★★

怎么选：

- 个人项目，想快速体验→ Aider。30 分钟上手，Repo Map 天然实现"渐进式披露"，自动 lint + test 构成反馈回路。
pip install aider-chat
cd your-project && aider --model gpt-4o
- 团队项目，需要流程控制→ Cline + Superpowers。Plan/Act 模式让你审批 Agent 的计划再执行，Superpowers 强制 TDD。
VS Code 中安装 Cline 扩展
然后在 Claude Code 中安装 Superpowers
claude /plugin install superpowers@claude-plugins-official
- 生产级，需要完全隔离→ OpenHands。Docker 沙箱确保 Agent 操作不影响主环境，模型无关。
pip install openhands-ai
openhands --model gpt-4o

5.3 Superpowers 深度解析

Superpowers 值得单独讲，因为它是目前 Harness Engineering 思想最完整的开源实现。

核心流程：

1. Brainstorming 苏格拉底式追问，确保需求清晰 "你说的 X 功能是指...？有没有考虑过 Y 方案？"
2. Git Worktree 自动创建隔离分支，不污染主分支
3. Writing Plans 拆解为 2-5 分钟的微任务清单 每个任务有明确的输入/输出/验收标准
4. Subagent-Driven Development 每个微任务分配独立子 Agent 天然避免上下文污染和任务干扰
5. Test-Driven Development（强制！） RED → GREEN → REFACTOR ⚠️ 如果 Agent 先写代码再写测试，代码会被删除
6. Code Review 自动按规范检查代码质量
7. Finishing Branch 验证所有测试通过 → 合并 → 清理 worktree

杀手特性——强制 TDD：

大多数 Agent 工具的问题是 Agent 会"先写实现再补测试"，甚至跳过测试。Superpowers 用一个极端手段解决这个问题：如果检测到 Agent 在测试之前写了实现代码，直接删除那段代码，强制从测试开始。

这听起来激进，但效果显著：测试先行确保了每一段代码都有验证，这正是 Harness Engineering "Verify" 环节的核心。

5.4 学习资源

资源	类型	适合谁	链接
deusyu/harness-engineering	GitHub 学习指南	从零入门	GitHub
harness-engineering.ai	知识图谱（883 实体）	快速了解生态全貌	Website
OpenAI 原文	博客	理解原始思想	OpenAI

六、实战案例：从零搭建一个 Harness 环境（30 分钟）

以一个 Next.js 项目为例，走完 Phase 1 + Phase 2 的核心步骤。

Step 1: 初始化项目结构（5 分钟）

```
# 创建 docs 目录结构
mkdir -p docs/{architecture,conventions,design,plans,reference}

# 创建 AGENTS.md
cat > AGENTS.md << 'EOF'
# AGENTS.md

## 项目简介
这是一个任务管理 Web 应用，基于 Next.js 14 + PostgreSQL + Prisma。

## 快速导航
| 你想做什么 | 去哪里看 |
|-----|-----|
| 了解系统架构 | docs/architecture/overview.md |
| 了解编码规范 | docs/conventions/README.md |
| 了解当前任务 | docs/plans/current-sprint.md |

## 硬性规则
1. 依赖方向: types/ → lib/ → services/ → app/
2. 禁止 console.log, 使用结构化日志
3. 单文件 ≤ 300 行
4. 新功能必须有测试
5. API 调用使用统一的 apiClient
EOF
```

Step 2: 写架构文档（5 分钟）

```
cat > docs/architecture/overview.md << 'EOF'
# 系统架构

## 分层结构

src/
|—— types/ # 纯类型定义（不依赖任何层）
```

```

|—— lib/ # 工具函数和基础设施（只依赖 types/）
| |—— api-client.ts
| |—— logger.ts
| |—— providers.ts
|—— services/ # 业务逻辑（依赖 types/ 和 lib/）
|—— app/ # Next.js 路由和页面（依赖所有层）
|—— components/ # UI 组件（只依赖 types/ 和 lib/providers）

```

依赖规则

- types/ → 不依赖任何层
- lib/ → 只依赖 types/
- services/ → 依赖 types/ 和 lib/
- components/ → 依赖 types/ 和 lib/providers
- app/ → 可以依赖所有层

EOF

Step 3: 配置 Linter 约束（10 分钟）

```

# 在 eslint.config.js 中添加自定义规则
cat > eslint.config.custom.js << 'EOF'
// 自定义 Harness 规则
module.exports = {
  rules: {
    // 禁止 UI 组件直接引用 services
    'no-restricted-imports': ['error', {
      patterns: [{
        group: ['../../services/*', '../services/*', '@services/*'],
        importNames: undefined,
        message: '❌ 组件层不能直接引用 Service 层。✅ FIX: 在 app/ 路由中调用 service, 通过 props 传递数据给组件。'
      }]
    }],
    'no-console': ['error', {
      allow: ['warn', 'error']
    }],
    'max-lines': ['error', { max: 300, skipBlankLines: true, skipComments: true }]
  }
};
EOF

```

Step 4: 配置 CI（8 分钟）

```

mkdir -p .github/workflows
cat > .github/workflows/harness-checks.yml << 'EOF'
name: Harness Checks
on: [pull_request]

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20' }
      - run: npm ci
      - run: npx tsc --noEmit
      - run: npm run lint
      - run: npm test -- --coverage --passWithNoTests
      - name: File size check
        run: |
          find src/ -name '*.ts' -o -name '*.tsx' | while read file; do
            lines=$(wc -l < "$file")

```

```
if [ "$lines" -gt 300 ]; then
    echo "❌ $f: $lines lines (max 300)"
    echo "✅ FIX: Split into smaller modules"
    exit 1
fi
done
EOF
```

Step 5: 验证（2 分钟）

让 Agent 执行一个简单任务，验证 Harness 是否生效：

任务：在 `services/` 中添加一个 `getUserById` 函数。

要求：有类型定义、错误处理、单元测试。

预期：

- ✅ Agent 在 `types/` 中定义 `User` 类型
- ✅ Agent 在 `services/` 中实现函数
- ✅ Agent 写了对应测试
- ❌ 如果 Agent 在 `components/` 中直接 `import services/`，Linter 报错
- ❌ 如果文件超 300 行，Linter 报错

七、踩坑指南：我们犯过的错

7.1 AGENTS.md 写太长

症状：Agent 的输出质量反而下降了，经常忽略部分规则。

原因：上下文窗口被指令文件占满，留给“正事”的空间不够。

解法：砍到 50-100 行。如果超过，把内容移到 `docs/` 并在 `AGENTS.md` 中只留链接。

7.2 Linter 规则太多，Agent 陷入死循环

症状：Agent 修了一个 Linter 错误，引入了另一个，反复循环。

原因：规则之间有冲突，或者修复建议不够具体。

解法：

- 逐条添加 Linter 规则，每加一条都让 Agent 试跑一遍
- 确保每条规则的 FIX 信息给出具体代码片段，而不是模糊的建议
- 互相冲突的规则只保留一条

7.3 架构约束太严，阻碍合理的跨层调用

症状：某些合理的代码模式被 CI 拦截，团队开始绕过规则。

解法：

- 设置“豁免白名单”机制，在特定目录或文件中可以显式声明例外
- 定期回顾约束规则，根据实际开发需要调整

```
// eslint-disable-next-line no-restricted-imports -- 此
```

```
import { dbClient } from '@repo/client';
```

7.4 Doc-gardening 没人管，文档比不写还误导

症状：Agent 参考了过时文档，写出的代码基于错误的假设。

解法：

- 在 CI 中加文档新鲜度检查（见上面 3.3 节的 CI 配置）
- 每两周跑一次 doc-gardening Agent
- 设计文档加 status 字段，过期的标记为 deprecated

7.5 过度依赖 Agent，忘了"审查环境"

症状：Agent 产出质量下降，团队没有及时发现。

解法：每周花 30 分钟做一次"环境审查"：

- 最近一周的 CI 失败率是否上升？
- Linter 规则是否覆盖了新出现的 bad pattern？
- 有没有新的架构约束需要加？
- AGENTS.md 和 docs/ 是否跟代码库一致？

八、总结：Harness Engineering 落地清单

□ Phase 1：信息层

- 创建 AGENTS.md (50–100 行，地图模式)
- 建立 docs/ 结构化目录
- 编写架构文档 + 编码规范文档
- 设计文档模板（含 Status 标记）

□ Phase 2：约束层

- 配置分层架构的 Linter 规则
- 每条 Linter 错误包含  问题 +  修复 +  文档
- CI 管线：类型检查 + Lint + 测试 + 覆盖率 + 架构约束
- 把团队口头约定翻译成机械规则

□ Phase 3：自动化层（可选，长期项目推荐）

- Git Worktree 隔离验证脚本
- 后台清理 Agent 定时任务
- 可观测性堆栈（日志 + 指标）
- 文档新鲜度自动检查

□ 持续维护

- 每周 30 分钟"环境审查"
- 每月回顾并更新 Linter 规则
- 每两周运行 doc-gardening Agent

记住：Harness Engineering 的核心不是搭建复杂的基础设施，而是一个简单的闭环——约束 → 告知 → 验证 → 纠正。从 AGENTS.md 和一条 Linter 规则开始，比什么都不做强一百倍。

参考资料

- 上篇：[Harness Engineering：当人类不再写代码，软件工程反而更"工程"了](#)
- OpenAI：[Harness engineering: leveraging Codex in an agent-first world](#)
- [Superpowers](#) — Agent 技能框架 (127k ⭐)
- [OpenHands](#) — AI 软件开发平台 (50k+ ⭐)
- [Aider](#) — 终端 AI 结对编程 (30k+ ⭐)
- [Cline](#) — VS Code Agent (40k+ ⭐)
- [SWE-agent](#) — 自主修复 Agent (15k+ ⭐)
- [deusyu/harness-engineering](#) — 学习指南 (310 ⭐)
- [harness-engineering.ai](#) — 知识图谱 (883 实体)

发布于 2026-04-02 16:06 · 北京



搭子 DuMate
让AI变成搭子

日程规划 数据分析
代码生成 文档生成

别再撸电子猫了，下一代“电子龙虾”搭子 DuMate 才是真搭子

AI兴趣党的快乐，就是有个能替你熬夜的龙虾级搭子。 [查看详情](#)

百度智能云 的广告



理性发言，友善互动

5 条评论

默认 最新

- 

落叶

刚好在学这个，感谢分享

04-10 · 四川

回复 喜欢
- 

我是迷茫狗

学完这个的时间 我感觉我已经 vibe 了一百个项目出来了🤔

04-07 · 江苏

回复 喜欢
- 

1watch2out3limar

最近正好在学 harness, 我愿称老哥为及时雨

04-07 · 上海

回复 喜欢
- 

栋栋

老哥，你上篇链接挂了

04-05 · 浙江

回复 喜欢
- 

AI4S 作者

直接去主页文章搜吧，或者公众号。知乎 md 文档排版也不太行， [Harness Engineering：当人类不再写代码，软件工程反而更"工程"了](#)

04-05 · 中国台湾

回复 喜欢

